

```
!pip install -q transformers torch sentencepiece
```

```
import torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
PyTorch version: 2.8.0+cu126
CUDA available: True
Using device: cuda
```

```
from transformers import MarianMTModel, MarianTokenizer

model_name = "Helsinki-NLP/opus-mt-fr-en"

# Load tokenizer and model
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name).to(device)

print("Model loaded ✅")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as the secret `HF_TOKEN` in your Colab secrets.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

```
warnings.warn(
tokenizer_config.json: 100% 42.0/42.0 [00:00<00:00, 4.23kB/s]
source.spm: 100% 802k/802k [00:00<00:00, 9.38MB/s]
target.spm: 100% 778k/778k [00:00<00:00, 60.9MB/s]
vocab.json: 1.34M/? [00:00<00:00, 55.5MB/s]
config.json: 1.42k/? [00:00<00:00, 156kB/s]
```

```
/usr/local/lib/python3.12/dist-packages/transformers/models/arian/tokenization_arian.py:175: UserWarning: Recommended: pip install sacremoses.
warnings.warn("Recommended: pip install sacremoses.")
pytorch_model.bin: 100% 301M/301M [00:03<00:00, 120MB/s]
generation_config.json: 100% 293/293 [00:00<00:00, 25.3kB/s]
model.safetensors: 0% 0.00/301M [00:00<?, ?B/s]
Model loaded ✅
```

```
french_text = "Le changement climatique est l'un des plus grands défis de notre époque."
```

```
# Tokenize and translate
inputs = tokenizer(french_text, return_tensors="pt", padding=True).to(device)
translated_ids = model.generate(**inputs)
```

```
# Decode to English
english_text = tokenizer.decode(translated_ids[0], skip_special_tokens=True)
print("French:", french_text)
print("English:", english_text)
```

```
French: Le changement climatique est l'un des plus grands défis de notre époque.
English: Climate change is one of the greatest challenges of our time.
```

```
french_texts = [
    "Le président a annoncé de nouvelles mesures économiques pour stimuler la croissance.",
    "La conférence sur le climat réunira des leaders mondiaux à Genève la semaine prochaine."
]
```

```
# Tokenize batch
inputs = tokenizer(french_texts, return_tensors="pt", padding=True, truncation=True).to(device)
translated_ids = model.generate(**inputs)
```

```
# Decode each translation
english_texts = [tokenizer.decode(t, skip_special_tokens=True) for t in translated_ids]
```

```
for i, (fr, en) in enumerate(zip(french_texts, english_texts), 1):
    print(f"--- Text {i} ---")
    print("French:", fr)
    print("English:", en, "\n")
```

--- Text 1 ---

French: Le président a annoncé de nouvelles mesures économiques pour stimuler la croissance.
English: The President announced new economic measures to stimulate growth.

--- Text 2 ---

French: La conférence sur le climat réunira des leaders mondiaux à Genève la semaine prochaine.
English: The climate conference will bring together world leaders in Geneva next week.

```
def chunk_and_translate(text, tokenizer, model, chunk_size=512):
    token_ids = tokenizer.encode(text)
    chunks = [token_ids[i:i+chunk_size] for i in range(0, len(token_ids), chunk_size)]

    translations = []
    for chunk in chunks:
        chunk_text = tokenizer.decode(chunk, skip_special_tokens=True)
        inputs = tokenizer(chunk_text, return_tensors="pt", padding=True).to(device)
        translated_ids = model.generate(**inputs)
        translations.append(tokenizer.decode(translated_ids[0], skip_special_tokens=True))
    return " ".join(translations)

# Example long text
long_french_text = """
Le gouvernement français a annoncé un plan ambitieux pour réduire les émissions de carbone de 50% d'ici 2035.
Le plan inclut des investissements massifs dans les énergies renouvelables et des subventions pour les véhicules électriques.
Des experts mettent en garde sur la nécessité de mesures concrètes pour atteindre ces objectifs ambitieux.
"""

english_translation = chunk_and_translate(long_french_text, tokenizer, model)
print("English Translation:\n", english_translation)
```

y and subsidies for electric vehicles. Experts warn about the need for concrete measures to achieve these ambitious objectives.

```
with open("translated_document.txt", "w", encoding="utf-8") as f:
    f.write(english_translation)
print("Saved as translated_document.txt ✓")
```

Saved as translated_document.txt ✓